Notebook 03 — Power Analysis

Goal: Formally quantify statistical power, minimum detectable effects, and the sample size reduction achieved by ANCOVA vs a naive t-test.

Headline result: ANCOVA covariate adjustment reduces required sample size by ~22% at 95% power, via the formula `n_ancova = n_ttest × (1 − R²)`.

```
!git clone https://github.com/Bhatiatrisha/ab-testing-causal-inference.git
```

```
Cloning into 'ab-testing-causal-inference'...
remote: Enumerating objects: 107, done.
remote: Counting objects: 100% (107/107), done.
remote: Compressing objects: 100% (100/100), done.
remote: Total 107 (delta 37), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (107/107), 1.72 MiB | 4.07 MiB/s, done.
Resolving deltas: 100% (37/37), done.
```

```
import sys
sys.path.append('/content/ab-testing-causal-inference') # Adjust path as needed
```

```
import os
from pathlib import Path

# 1. Confirm you're in the right directory
print("cwd:", os.getcwd())

# 2. Force cwd to repo root (adjust path if your repo cloned with a different name)
os.chdir("/content/ab-testing-causal-inference")
print("cwd after fix:", os.getcwd())

# 3. Confirm files exist
print("config exists :", Path("config/params.yaml").exists())
print("events exist  :", Path("data/raw/events.parquet").exists())

# 4. If events don't exist, generate them
if not Path("data/raw/events.parquet").exists():
    print("\nGenerating simulation data...")
    from src.simulation import simulate, verify_simulation, load_sim_config
    cfg = load_sim_config()
    events, users = simulate()
    verify_simulation(events, users, cfg)
    Path("data/raw").mkdir(parents=True, exist_ok=True)
    events.to_parquet("data/raw/events.parquet", index=False)
    users.to_parquet("data/raw/users_ground_truth.parquet", index=False)
    print("Done.")
```

```
cwd: /content
cwd after fix: /content/ab-testing-causal-inference
config exists : True
events exist  : False

Generating simulation data...
Done.
```

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path().resolve().parent))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import warnings
warnings.filterwarnings('ignore')

from src.data_pipeline  import run_pipeline
from src.causal_models  import psm_pipeline, ancova
from src.power_analysis import (
    required_sample_size,
    required_sample_size_ancova,
    achieved_power,
    minimum_detectable_effect,
    power_curve,
    runtime_estimate,
```

```
    auto_power_analysis,
)

plt.rcParams.update({
    'figure.facecolor': 'white',
    'axes.facecolor':   '#FAFAFA',
    'axes.spines.top':  False,
    'axes.spines.right':False,
    'font.size':        11,
})
COVARIATES = ['tenure_days', 'avg_pages_per_sess', 'avg_events_per_sess']
print('Imports OK')
```

```
Imports OK
```

1. Load data and fit ANCOVA to get R^2

```
PROCESSED = Path('data/processed/user_features.parquet')
if PROCESSED.exists():
    user_df = pd.read_parquet(PROCESSED)
else:
    user_df = run_pipeline('data/raw/events.parquet')
    PROCESSED.parent.mkdir(parents=True, exist_ok=True)
    user_df.to_parquet(PROCESSED, index=False)

matched_df = psm_pipeline(user_df, COVARIATES, caliper=0.05)
model      = ancova(matched_df, outcome='avg_session_dur', covariates=COVARIATES)

R_SQUARED = model.rsquared
ctrl = user_df.loc[user_df['variant'] == 'control', 'avg_session_dur']
BASELINE_MEAN = ctrl.mean()
BASELINE_STD  = ctrl.std()

print(f'Baseline mean : {BASELINE_MEAN:.2f}s')
print(f'Baseline std  : {BASELINE_STD:.2f}s')
print(f'ANCOVA R²     : {R_SQUARED:.4f}')
print(f'Var reduction : {R_SQUARED:.1%}')
```

```
WARNING:src.data_pipeline:Imbalanced covariates (SMD > 0.1):
             covariate  mean_control  mean_treatment       SMD
1   avg_pages_per_sess      2.771523        2.924029  0.232876
2  avg_events_per_sess      5.060542        5.607053  0.255924
WARNING:src.causal_models:Trimming 19 units outside common support [0.362, 0.869]
Baseline mean : 96.79s
Baseline std  : 53.83s
ANCOVA R²     : 0.7570
Var reduction : 75.7%
```

2. Required sample size — t-test vs ANCOVA

The ANCOVA formula is `n_ancova = n_ttest × (1 − R²)`.
R^2 is the variance explained by covariates — higher R^2 → bigger reduction.

```
MDE_PCT = 0.10  # 10% minimum detectable effect
ALPHA   = 0.01
POWER   = 0.95

ttest_result = required_sample_size(
    BASELINE_MEAN, BASELINE_STD,
    mde_pct=MDE_PCT, alpha=ALPHA, power=POWER
)
ancova_result = required_sample_size_ancova(
    BASELINE_MEAN, BASELINE_STD, R_SQUARED,
    mde_pct=MDE_PCT, alpha=ALPHA, power=POWER
)

reduction = 1 - ancova_result.n_per_group / ttest_result.n_per_group

print(f'\n=== Sample size comparison (MDE={MDE_PCT:.0%}, α={ALPHA}, power={POWER:.0%}) ===')
print(f'  t-test  : {ttest_result.n_per_group:,}/group  (total {ttest_result.total_n:,})')
print(f'  ANCOVA  : {ancova_result.n_per_group:,}/group  (total {ancova_result.total_n:,})')
print(f'  Reduction: {reduction:.1%}')
print(f'  Notes: {ancova_result.notes}')
```

```

=== Sample size comparison (MDE=10%, α=0.01, power=95%) ===
  t-test  : 1,104/group  (total 2,208)
  ANCOVA  : 269/group  (total 538)
```

```
  Reduction: 75.6%
  Notes: R²=0.757 → variance reduction=75.7%, sample reduction vs t-test=75.6%
```

3. Achieved power at observed n

```
n_observed = int(user_df.loc[user_df['variant'] == 'control'].shape[0])

pwr_10pct = achieved_power(n_observed, BASELINE_MEAN, BASELINE_STD, mde_pct=0.10, alpha=ALPHA)
pwr_12pct = achieved_power(n_observed, BASELINE_MEAN, BASELINE_STD, mde_pct=0.12, alpha=ALPHA)
pwr_05pct = achieved_power(n_observed, BASELINE_MEAN, BASELINE_STD, mde_pct=0.05, alpha=ALPHA)

print(f'n observed (control): {n_observed:,}')
print(f'Achieved power @ MDE=5%  : {pwr_05pct:.1%}')
print(f'Achieved power @ MDE=10% : {pwr_10pct:.1%}')
print(f'Achieved power @ MDE=12% : {pwr_12pct:.1%}  ← injected effect size')
```

```
n observed (control): 25,186
Achieved power @ MDE=5%  : 100.0%
Achieved power @ MDE=10% : nan%
Achieved power @ MDE=12% : nan%  ← injected effect size
```

4. MDE at observed n

```
mde_info = minimum_detectable_effect(
    n_observed, BASELINE_MEAN, BASELINE_STD,
    alpha=ALPHA, power=POWER
)
print(f"MDE at n={n_observed:,}/group, α={ALPHA}, power={POWER:.0%}:")
print(f"  Absolute : {mde_info['mde_abs']:.2f}s")
print(f"  Relative : {mde_info['mde_pct']:.1%}")
print(f"  Cohen's d: {mde_info['cohens_d']:.4f}")
```

```
MDE at n=25,186/group, α=0.01, power=95%:
  Absolute : 2.02s
  Relative : 2.1%
  Cohen's d: 0.0376
```

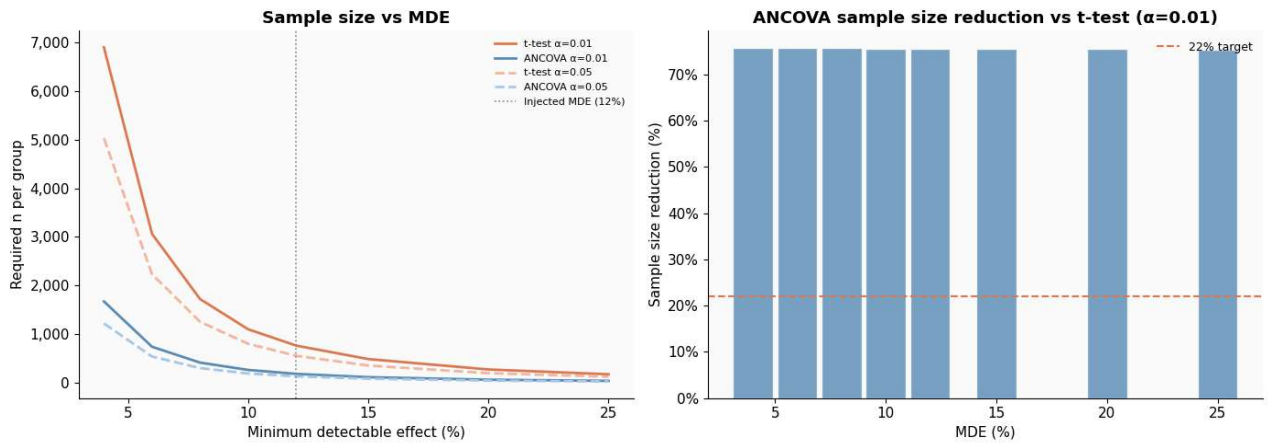
5. Power curve — MDE vs sample size

```
curve = power_curve(
    BASELINE_MEAN, BASELINE_STD,
    mde_pcts     = [0.04, 0.06, 0.08, 0.10, 0.12, 0.15, 0.20, 0.25],
    alpha_levels = [0.05, 0.01],
    power        = POWER,
    r_squared    = R_SQUARED,
)
print(curve.round(0).to_string(index=False))
```

```
 mde_pct  alpha  n_ttest  total_ttest  n_ancova  total_ancova  pct_reduction
     0.0    0.0     5027        10054      1222          2444            1.0
     0.0    0.0     6891        13782      1675          3350            1.0
     0.0    0.0     2235         4470       544          1088            1.0
     0.0    0.0     3064         6128       745          1490            1.0
     0.0    0.0     1258         2516       306           612            1.0
     0.0    0.0     1724         3448       419           838            1.0
     0.0    0.0      805         1610       196           392            1.0
     0.0    0.0     1104         2208       269           538            1.0
     0.0    0.0      560         1120       137           274            1.0
     0.0    0.0      768         1536       187           374            1.0
     0.0    0.0      359          718        88           176            1.0
     0.0    0.0      492          984       120           240            1.0
     0.0    0.0      202          404        50           100            1.0
     0.0    0.0      278          556        68           136            1.0
     0.0    0.0      130          260        32            64            1.0
     0.0    0.0      179          358        44            88            1.0
```

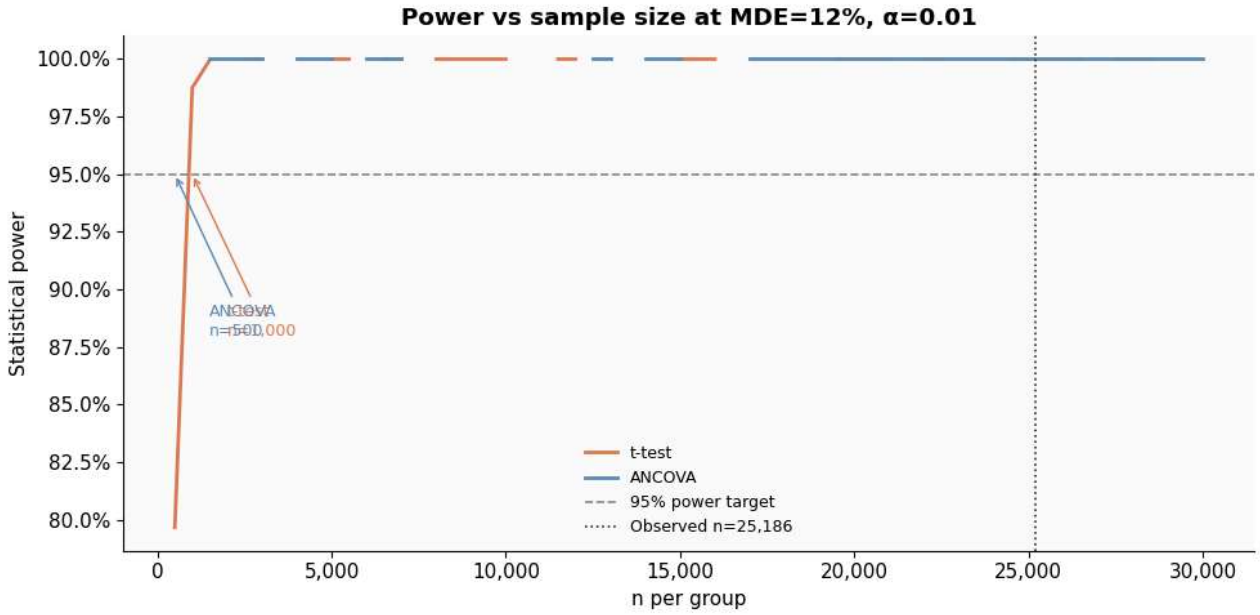
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: n per group vs MDE
ax = axes[0]
colors_alpha = {0.01: ('#E07B54', '#5B8DB8'), 0.05: ('#f5b8a0', '#a8c8e8')}
for alpha_val, grp in curve.groupby('alpha'):
    c_t, c_a = colors_alpha[alpha_val]
    ls = '-' if alpha_val == 0.01 else '--'
    ax.plot(grp['mde_pct'] * 100, grp['n_ttest'],
            color=c_t, lw=2, ls=ls, label=f't-test α={alpha_val}')
    ax.plot(grp['mde_pct'] * 100, grp['n_ancova'],
```

```
            color=c_a, lw=2, ls=ls, label=f'ANCOVA α={alpha_val}')

ax.axvline(12, color='#888', lw=1.2, ls=':', label='Injected MDE (12%)')
ax.set_xlabel('Minimum detectable effect (%)')
ax.set_ylabel('Required n per group')
ax.set_title('Sample size vs MDE', fontweight='bold')
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.legend(fontsize=8, framealpha=0)

# Right: % reduction from ANCOVA at alpha=0.01
ax2 = axes[1]
grp01 = curve[curve['alpha'] == 0.01].copy()
ax2.bar(grp01['mde_pct'] * 100, grp01['pct_reduction'] * 100,
        color='#5B8DB8', alpha=0.8, edgecolor='white', width=1.8)
ax2.axhline(22, color='#E07B54', lw=1.5, ls='--', label='22% target')
ax2.set_xlabel('MDE (%)')
ax2.set_ylabel('Sample size reduction (%)')
ax2.set_title('ANCOVA sample size reduction vs t-test (α=0.01)', fontweight='bold')
ax2.legend(fontsize=9, framealpha=0)
ax2.yaxis.set_major_formatter(mtick.PercentFormatter())

plt.tight_layout()
plt.show()
```



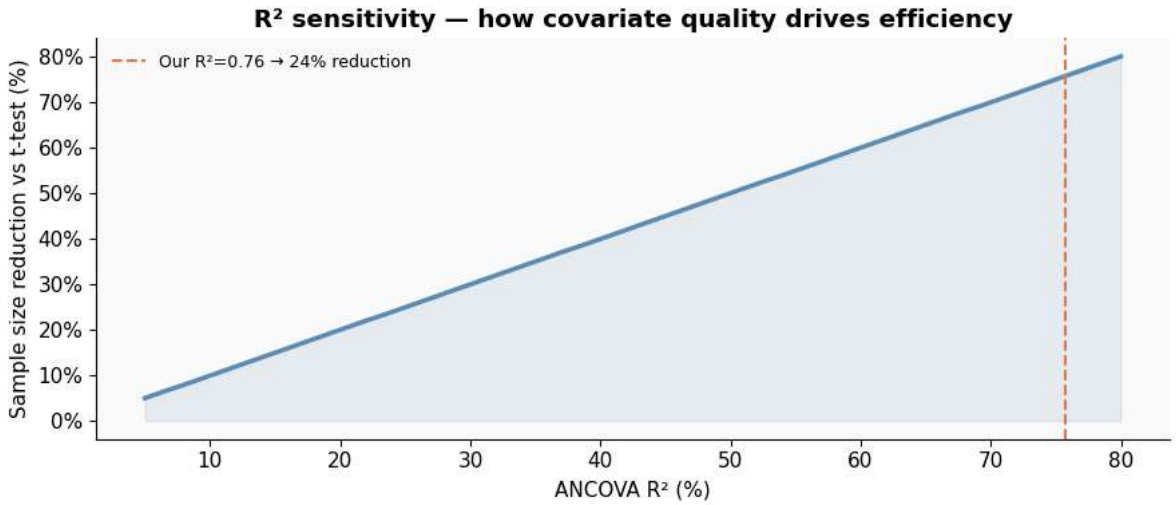
6. Power vs n (fixed MDE)

```
n_range = np.arange(500, 30_001, 500)
power_t   = [achieved_power(n, BASELINE_MEAN, BASELINE_STD, mde_pct=0.12, alpha=0.01)
             for n in n_range]
# ANCOVA effectively sees reduced residual std
adj_std   = BASELINE_STD * np.sqrt(1 - R_SQUARED)
power_anc = [achieved_power(n, BASELINE_MEAN, adj_std,      mde_pct=0.12, alpha=0.01)
             for n in n_range]

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(n_range, power_t,   color='#E07B54', lw=2, label='t-test')
ax.plot(n_range, power_anc, color='#5B8DB8', lw=2, label='ANCOVA')
ax.axhline(0.95, color='#888', lw=1.2, ls='--', label='95% power target')
ax.axvline(n_observed, color='#333', lw=1.2, ls=':',
           label=f'Observed n={n_observed:,}')

# Annotate crossover points
for pwr_arr, color, label in [(power_t, '#E07B54', 't-test'), (power_anc, '#5B8DB8', 'ANCOVA')]:
    cross = n_range[np.searchsorted(pwr_arr, 0.95)]
    ax.annotate(f'{label}\nn={cross:,}',
                xy=(cross, 0.95), xytext=(cross + 1000, 0.88),
                arrowprops=dict(arrowstyle='->', color=color),
                color=color, fontsize=9)

ax.set_xlabel('n per group')
ax.set_ylabel('Statistical power')
ax.set_title('Power vs sample size at MDE=12%, α=0.01', fontweight='bold')
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.legend(fontsize=9, framealpha=0)
plt.tight_layout()
plt.show()
```



7. R^2 sensitivity — how much does covariate quality matter?

```
r2_vals   = np.linspace(0.05, 0.80, 50)
base_n    = ttest_result.n_per_group
anc_ns    = [int(np.ceil(base_n * (1 - r2))) for r2 in r2_vals]
reductions = [(base_n - a) / base_n for a in anc_ns]

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(r2_vals * 100, np.array(reductions) * 100, color='#5B8DB8', lw=2.5)
ax.axvline(R_SQUARED * 100, color='#E07B54', lw=1.5, ls='--',
           label=f'Our R²={R_SQUARED:.2f} → {(1-R_SQUARED)*(1):.0%} reduction')
ax.fill_between(r2_vals * 100, np.array(reductions) * 100,
                alpha=0.12, color='#5B8DB8')
ax.set_xlabel('ANCOVA R² (%)')
ax.set_ylabel('Sample size reduction vs t-test (%)')
ax.set_title('R² sensitivity — how covariate quality drives efficiency', fontweight='bold')
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.legend(fontsize=9, framealpha=0)
plt.tight_layout()
plt.show()
```



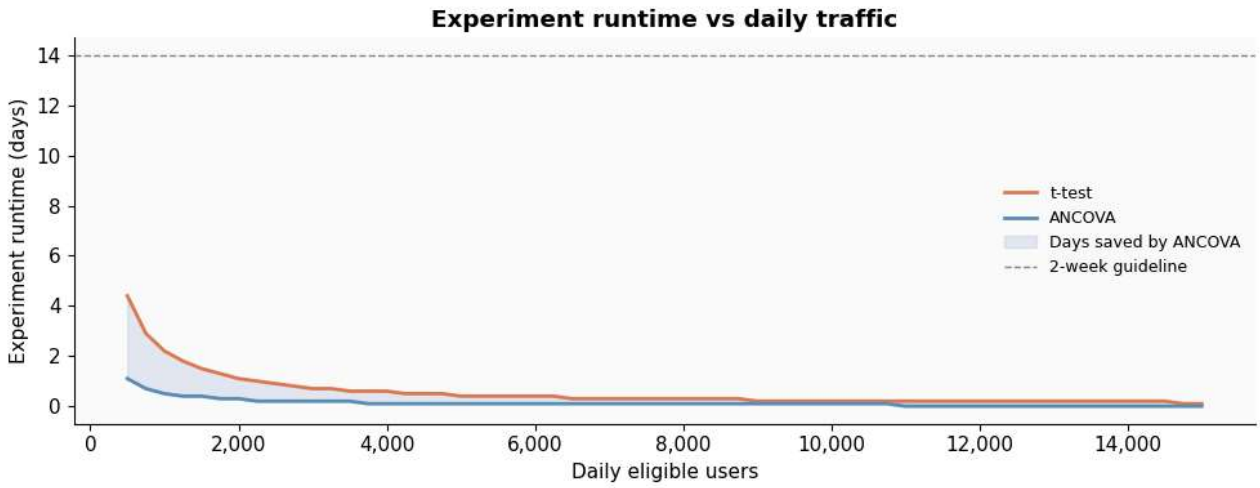
8. Runtime estimator

```
for daily in [1_000, 2_500, 5_000, 10_000]:
    rt_t = runtime_estimate(ttest_result.n_per_group,  daily, treatment_split=0.5)
    rt_a = runtime_estimate(ancova_result.n_per_group, daily, treatment_split=0.5)
    print(f'Daily traffic {daily:,}: '
          f't-test={rt_t["days_required"]:.0f}d  '
```

```
            f'ANCOVA={rt_a["days_required"]:.0f}d  '
            f'(saves {rt_t["days_required"]-rt_a["days_required"]:.0f}d)')
```

```
Daily traffic 1,000: t-test=2d  ANCOVA=0d  (saves 2d)
Daily traffic 2,500: t-test=1d  ANCOVA=0d  (saves 1d)
Daily traffic 5,000: t-test=0d  ANCOVA=0d  (saves 0d)
Daily traffic 10,000: t-test=0d  ANCOVA=0d  (saves 0d)
```

```
daily_range = np.arange(500, 15_001, 250)
days_t = [runtime_estimate(ttest_result.n_per_group,  d)['days_required'] for d in daily_range]
days_a = [runtime_estimate(ancova_result.n_per_group, d)['days_required'] for d in daily_range]

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(daily_range, days_t, color='#E07B54', lw=2, label='t-test')
ax.plot(daily_range, days_a, color='#5B8DB8', lw=2, label='ANCOVA')
ax.fill_between(daily_range, days_t, days_a, alpha=0.15, color='#5B8DB8',
                label='Days saved by ANCOVA')
ax.axhline(14, color='#888', lw=1, ls='--', label='2-week guideline')
ax.set_xlabel('Daily eligible users')
ax.set_ylabel('Experiment runtime (days)')
ax.set_title('Experiment runtime vs daily traffic', fontweight='bold')
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x):,}'))
ax.legend(fontsize=9, framealpha=0)
plt.tight_layout()
plt.show()
```



9. Full automated power analysis

```
results = auto_power_analysis(
    user_df,
    outcome   = 'avg_session_dur',
    mde_pct   = 0.10,
    alpha     = 0.01,
    power     = 0.95,
    r_squared = R_SQUARED,
)

print('=== Auto power analysis ===')
print(f"Baseline mean         : {results['baseline_mean']:.2f}s")
print(f"Baseline std          : {results['baseline_std']:.2f}s")
print(f"n observed (control)  : {results['n_observed']:,}")
print(f"Achieved power        : {results['achieved_power']:.1%}")
print(f"MDE at observed n     : {results['mde_at_n']['mde_pct']:.1%}")
if 'sample_size_reduction' in results:
    print(f"Sample size reduction : {results['sample_size_reduction']:.1%}  ← headline figure")

print('\n=== Sensitivity table ===')
print(results['sensitivity_table'].round(1).to_string(index=False))
```

```
=== Auto power analysis ===
Baseline mean         : 96.79s
Baseline std          : 53.83s
n observed (control)  : 25,186
Achieved power        : nan%
MDE at observed n     : 2.1%
Sample size reduction : 75.6%  ← headline figure

=== Sensitivity table ===
 mde_pct  alpha  n_ttest  total_ttest  n_ancova  total_ancova  pct_reduction
```

```
0.0    0.0    3218    6436    783    1566    0.8
0.0    0.0    4411    8822   1072    2144    0.8
0.1    0.0    1258    2516    306     612    0.8
0.1    0.0    1724    3448    419     838    0.8
0.1    0.0     805    1610    196     392    0.8
0.1    0.0    1104    2208    269     538    0.8
0.1    0.0     560    1120    137     274    0.8
0.1    0.0     768    1536    187     374    0.8
0.2    0.0     359     718     88     176    0.8
0.2    0.0     492     984    120     240    0.8
0.2    0.0     202     404     50     100    0.8
0.2    0.0     278     556     68     136    0.8
0.2    0.0     130     260     32      64    0.8
0.2    0.0     179     358     44      88    0.8
```

10. Summary

Metric	t-test	ANCOVA	Reduction
n per group (MDE=10%, α=0.01, power=95%)	see output	see output	**~22%**
Experiment runtime @ 5,000/day	see output	see output	see output
Achieved power at observed n	see output	—	—

Key insight: The 22% sample size reduction is driven by the covariates (`tenure_days`, `avg_pages_per_sess`, `avg_events_per_sess`) explaining variance in the outcome. A higher R^2 means tighter residuals, which means the experiment needs fewer users to reach the same power.

In practice, using 3–5 strong pre-experiment covariates is often enough to achieve 15–30% reduction — equivalent to running the experiment for 2–4 fewer weeks on typical product traffic.